T1 旅游（travel）

知识点：字符串处理、条件判断

按四个条件依次判断

四个要求逐步增加限制：同车厢，再要求同排，再要求同侧，最后要求相邻。因此，后一项成立时，前面的条件也必须全部成立。

车厢号直接比较字符串；座位号的前两位是排号，最后一位是座位字母。字母 A, B, C 在一侧，D, F 在另一侧，所以可以用字母是否不大于 C 区分两侧。

同排、同侧时，相邻的座位只有 AB、BC、DF 三种组合，交换两人的位置不影响结果。先把两个字母按从小到大排列，再检查是否属于这三种即可。特别地，A, C 不相邻，而 D, F 虽然字母不连续，却是相邻座位。

用 b_1, b_2, b_3, b_4 依次记录四项是否成立，按顺序输出，成立输出 1，否则输出 0。输入字符串长度固定，时间、空间复杂度均为 $O(1)$。

满分参考代码

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    freopen("travel.in", "r", stdin);
    freopen("travel.out", "w", stdout);
    string c1, s1, c2, s2;
    cin >> c1 >> s1 >> c2 >> s2;
    char x = s1[2], y = s2[2];
    bool b1 = (c1 == c2);
    bool b2 = b1 && s1.substr(0, 2) == s2.substr(0, 2);
    bool b3 = b2 && ((x <= 'C') == (y <= 'C'));
    if (x > y) swap(x, y);
    bool near = (x == 'A' && y == 'B')
             || (x == 'B' && y == 'C')
             || (x == 'D' && y == 'F');
    bool b4 = b3 && near;
    cout << b1 << b2 << b3 << b4 << '\n';
    return 0;
}
```

部分分

每个测试点 5 分。性质 A 保证同车厢，B 保证同排，C 保证同侧；性质 D 保证第一个座位字母为 D，第二个为 F。各点组只是减少需要判断的条件，均可沿用上面的常数时间判断。

序号	条件与点组	点组分值	做法与复杂度
—	$1 \sim 4$：A、B、C	20 分	前三位直接为 1，第四位检查字母组合是否为 AB、BC 或 DF。时间、空间均为 $O(1)$。
—	$5 \sim 10$：A、B	30 分	前两位为 1；根据字母判断同侧与相邻。时间、空间均为 $O(1)$。
—	$11 \sim 14$：A	20 分	第一位为 1，再检查排号、同侧与相邻。时间、空间均为 $O(1)$。
—	$15 \sim 17$：D	15 分	不同车厢输出 `0000`；同车厢不同排输出 `1000`；同车厢同排时，D, F 同侧且相邻，输出 `1111`。时间、空间均为 $O(1)$。
—	$18 \sim 20$：无附加限制	15 分	依次检查车厢、排号、同侧与相邻，按正文的四个布尔值输出。时间、空间均为 $O(1)$，覆盖全部测试点。

T2 回文串（string）

知识点：双指针、分段处理

满分做法

回文串要求每一对镜像位置的字符相同。两个字符本来相同，就不用修改；不同，就至少改一个，而把其中一个改成另一个即可。各对镜像位置互不影响，所以答案就是**字符不同的镜像对数**。

用双指针从字符串两端向中间比较。由于同一段内的字符都相同，可以一次比较一批，而不必把字符串展开。

设当前两端位于第 l, r 段，a_l, a_r 记录这两段还没有处理的长度。当 $l < r$ 时，接下来的

$$t = \min(a_l, a_r)$$

对镜像位置，左边的字符都是 c_l，右边的字符都是 c_r。因此，若 $c_l \neq c_r$，就把答案加上 t；否则不用增加答案。随后将 a_l, a_r 都减去 t，哪一段用完了，就把对应指针向中间移动一段。

这样处理后，尚未比较的仍是原串中间的一个连续区间。若最后 $l = r$，剩余字符都来自同一段，本来就相同，不必再修改；若 $l > r$，则已经处理完毕。相邻两段即使字符相同，也不影响这个过程，无须预先合并。

每轮至少用完一段，因此总时间复杂度为 $O(n)$，保存各段需要 $O(n)$ 空间。原串长度最多为 2×10^{14}，答案最多为 10^{14}，要用 long long 保存。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 200000 + 10;
int n;
long long a[N];
char c[N];

int main(){
    freopen("string.in","r",stdin);
    freopen("string.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);
    cin >> n;
    for(int i = 1; i <= n; i++){
        cin >> a[i] >> c[i];
    }
    int l = 1,r = n;
    long long ans = 0;
    while(l < r){
        long long t = min(a[l],a[r]);
        if(c[l] != c[r]) ans += t;
        a[l] -= t;
        a[r] -= t;
        if(a[l] == 0) l++;
        if(a[r] == 0) r--;
    }
    cout << ans << '\n';
    return 0;
}
```

部分分思路

本题共 25 个测试点，每点 4 分。下表列出的分值是各点组的分值；$L = \sum a_i$ 表示展开后的串长。

序号	条件与点组	点组分值	做法与复杂度
1	测试点 $1 \sim 4$：$n \le 2$，$a_i \le 5$；其中点 1 满足 $n = 1$	16 分	一段时答案为 0。两段字母相同也为 0；不同则有 $\min(a_1, a_2)$ 对不同字符，答案就是这个数。读入与计算共 $O(n)$ 时间、$O(1)$ 额外空间。
2	测试点 $5 \sim 12$：均满足 $n \le 2000$，$a_i \le 50$	32 分	按每段的长度和字母展开字符串，再逐对比较镜像位置，统计不同的对数。此时 $L \le 10^5$，时间、空间均为 $O(n + L)$。
3	测试点 $13 \sim 16$：$n \le 2000$，$a_i \le 10^9$	16 分	枚举原串中的一段和反转串中的一段，计算它们在位置上的交集。字母不同就累加交集长度，最后除以 2。这样不必展开长串，时间 $O(n^2)$，空间 $O(n)$。
复用 2	测试点 $17, 18$：$n \le 2 \times 10^5$，$a_i = 1$	8 分	每段就是一个字符，展开后的 $L = n$，逐对比较即可。时间、空间均为 $O(n)$。
复用 2	测试点 $19 \sim 21$：$n \le 2 \times 10^5$，$a_i \le 500$	12 分	仍可展开后逐对比较。$L \le 10^8$，字符本身最多占约 100 MB；预先分配所需容量可避免扩容时复制。时间、空间均为 $O(n + L)$。
满分	测试点 $22 \sim 25$：无额外限制	16 分	沿正文的双指针做法，每次处理两端整批相同字符，时间、空间均为 $O(n)$。

第 3 种做法中，令 $b_i = \sum_{j=1}^{i} a_j$，$b_0 = 0$。第 i 段占据位置 $[b_{i-1}+1, b_i]$；反转后，第 j 段占据 $[L-b_j+1, L-b_{j-1}]$，所以用两区间左端点的最大值、右端点的最小值就能求出交集。每对不同的镜像字符在两个位置各被统计一次，因此最终除以 2。

展开法也覆盖测试点 $1 \sim 4$，合计覆盖 $1 \sim 12$、$17 \sim 21$，共 68 分；区间交法覆盖全部 $n \le 2000$ 的测试点，即 $1 \sim 16$，共 64 分。两者按数据规模选用，覆盖的是 $1 \sim 21$，共 84 分，重叠点组不能重复计分。

T3 按位与（and）

知识点：位运算、贪心

从高位到低位确定答案

按位与的某一位等于 1，当且仅当选出的 m 个数在这一位上全是 1。因此，要让答案包含若干个已经确定的 1 位，就必须找到至少 m 个数，**每个数都同时包含这些 1 位**。

把这些位组成的数记为 x。一个数 a_i 符合要求，恰好等价于

$$a_i \,\&\, x = x.$$

统计符合要求的位置数：若不少于 m，就能任选其中 m 个，使按位与的结果包含 x 中的全部 1 位；否则，这些位不可能同时保留。这里判断的是“包含这些位”，结果还可能有其他 1 位。

接下来考虑先确定哪一位。两个不同整数的大小由最高的不同位决定：第 b 位的权值为 2^b，比所有更低位的权值之和 $2^b - 1$ 还大。因此，应该从高位到低位，能让当前位为 1 就让它为 1。

用 ans 保存目前已经确定的 1 位，初始为 0。由于 $a_i < 2^{30}$，依次枚举 $b = 29, 28, \ldots, 0$，试着加入当前位，令

$$x = ans \mid (1 \ll b).$$

扫描数组，统计满足 $(a_i \,\&\, x) = x$ 的位置数。如果不少于 m，便令 $ans = x$；否则舍弃这一位，继续考虑更低位。

每次接受一位后，仍存在至少 m 个数同时满足当前要求。某一位被舍弃，说明任何保留已确定高位的方案都不能再包含这一位；以后增加要求也不会使它重新可行。低位的取值又无法补偿高位的损失，所以逐位选择结束后，ans 就是最大的按位与结果。

注意，每次都要检查整个 x。例如从 $6, 5, 3$ 中选两个数，虽然每一位分别都有两个数为 1，但并不存在两个数同时包含这三位，不能把三个独立统计的结果拼成 7。

共扫描数组 30 次，时间复杂度为 $O(30n)$；保存原数组，空间复杂度为 $O(n)$。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N = 200000 + 10;
int n,m,a[N];

int main(){
    freopen("and.in","r",stdin);
    freopen("and.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin >> n >> m;
    for(int i = 1; i <= n; i++) cin >> a[i];
    int ans = 0;
    for(int b = 29; b >= 0; b--){
        int x = ans | (1 << b);
        int cnt = 0;
        for(int i = 1; i <= n; i++){
            if((a[i] & x) == x) cnt++;
        }
        if(cnt >= m) ans = x;
    }
    cout << ans << '\n';
    return 0;
}
```

部分分思路

下表的分值是对应点组的分值，每个测试点为 5 分。

序号	条件与点组	点组分值	做法与复杂度
1	点 1～5：$n \le 20$	25 分	按下标递增的顺序搜索下一个选谁，维护已选数量和按位与；选满 m 个时更新答案，剩余位置不够时不再搜索。时间 $O(2^n)$，空间 $O(n)$。
2	点 6、7：$n \le 2000$，分别为 $m = 1$、$m = n$	10 分	只选一个时取最大值；全部都选时直接求所有数的按位与。一次扫描，时间 $O(n)$，额外空间 $O(1)$。
3	点 8～10：$n \le 2000$，$m = 2$	15 分	枚举两个不同的位置 $i < j$，取 $a_i \,\&\, a_j$ 的最大值。时间 $O(n^2)$，空间 $O(n)$。
4	点 11、12：$m = n - 1$，$n \le 2 \times 10^5$	10 分	恰好删去一个数。预处理前缀与后缀的按位与，删去第 i 个数后的结果为 $pre_{i-1} \,\&\, suf_{i+1}$，枚举 i 取最大值。时间、空间均为 $O(n)$。
5	点 13：性质 A，$a_i = 2^{k_i} - 1$	5 分	这些数的低位连续为 1，任取若干个数的按位与等于其中最小值。应选最大的 m 个数，答案是第 m 大。排序，时间 $O(n \log n)$，空间 $O(n)$。
6	点 14、15：性质 B，$n = 2^k$，$a_i = i - 1$	10 分	0 到 $n-1$ 恰好包含低 k 位的全部组合。候选答案有 t 个零位，就有 2^t 个数包含其全部一位。取最小的 t 使 $2^t \ge m$，再把零位放在最低的 t 位，得到最大答案 $n - 2^t$。含读入时间 $O(n + \log m)$，额外空间 $O(1)$。
7	点 16、17：$a_i < 4$	10 分	先统计每个数值出现的次数，再从大到小枚举候选答案 x。累加所有满足 $(v \,\&\, x) = x$ 的数值 v 的出现次数，第一个累计数不少于 m 的 x 就是答案。值域为 U 时，时间 $O(n + U^2)$，本档 $U = 4$；所给实现保存原数组，空间 $O(n + U)$。
复用 7	点 18、19：$a_i < 2^{10}$	10 分	仍按数值频次枚举候选，$U = 1024$，至多检查 U^2 对数值关系；时间 $O(n + U^2)$，空间 $O(n + U)$。
满分	点 20：无附加限制	5 分	按正文从第 29 位到第 0 位试置，每次扫描数组检查整组已选位是否可行，时间 $O(30n)$，空间 $O(n)$；同时覆盖全部测试点。

第 1 种搜索在 $m = 1$ 或 $m = n$ 时只需 $O(n)$，在 $m = 2$ 时只需 $O(n^2)$，因此也能覆盖点 6～10，合计保证点 1～10 的 50 分；第 2、3 种做法是这两类条件下更直接的写法。第 5 种做法在 $m = 1$ 时也会取到最大值，因此还覆盖点 6，合计保证 10 分。

前后缀按位与的空段应初始化为低 30 位全为 1 的数，即 $2^{30} - 1$，这样删去第一个或最后一个数时仍适用同一式子。性质 A、B 的结论只在对应保证下成立。

T4 纸牌游戏（card）

知识点：递推、贡献统计

满分做法

从第 i 张牌开始的一局游戏中，第一张牌的点数 $x = a_i$ 会一直留在桌面最左端：打出其他点数不能收走它，再次打出 x 也只是用新的 x 替换它。因此，可以先确定最终桌面最左边的牌，再考虑它右边留下什么。

记 $last[x]$ 为点数 x 在整个序列中最后出现的位置。打到第 $last[x]$ 张牌时，桌面会被清空后重新放上 x，此时只有这一张牌。后面不再出现 x，所以后续操作不会收走它；对其右侧的操作，恰好与从第 $last[x]+1$ 张牌开始、桌面初始为空的一局游戏相同。

于是，一局从 i 开始的游戏，可以拆成两件事：

1. 给点数 a_i 的答案贡献 1。
2. 继续处理从 $last[a_i]+1$ 开始的游戏；若这个位置是 $n+1$，则已经结束。

接下来要统计全部 n 局游戏。它们可能在拆分过程中到达同一个起点；到达这里以后，留下哪些牌完全相同，只需记录一共有多少局到达，不必分别处理。

用 f_i 表示拆分过程中到达起点 i 的游戏局数。最初，每个 $i \in [1,n]$ 都有一局游戏直接从这里开始，因此先令 $f_i = 1$。处理到 i 时，这 f_i 局都会留下 a_i，并继续到达 $last[a_i]+1$，所以更新：

$$ans[a_i] \leftarrow ans[a_i] + f_i, \qquad f_{last[a_i]+1} \leftarrow f_{last[a_i]+1} + f_i.$$

因为 $last[a_i] \geq i$，下一个起点一定在 i 的右侧，按 $i = 1, 2, \ldots, n$ 的顺序处理即可。这样处理 i 时，所有从左侧到达它的局数都已累计完成。$n+1$ 只是结束位置，不再处理。

每局游戏在留下点数 x 后都会跳过 x 的最后一次出现，后续不可能再次遇到 x，因此不会对同一点数重复计数。所有局的每张最终保留牌也都对应一次这样的贡献，得到的 ans 就是答案。

记录最后出现位置、累计贡献和输出答案共需 $O(n+m)$ 时间，空间复杂度为 $O(n+m)$。f_i 和 $ans[x]$ 都表示游戏局数，不超过 n，使用 int 即可。

满分参考代码

```cpp
#include<bits/stdc++.h>
using namespace std;

const int N = 500000 + 10;
const int M = 300000 + 10;
int n,m,a[N],f[N],last[M],ans[M];

int main(){
    freopen("card.in","r",stdin);
    freopen("card.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(0);

    cin >> n >> m;
    for(int i = 1; i <= n; i++){
        cin >> a[i];
        last[a[i]] = i;
        f[i] = 1;
    }
    for(int i = 1; i <= n; i++){
        int x = a[i];
        ans[x] += f[i];
        f[last[x] + 1] += f[i];
    }
    for(int x = 1; x <= m; x++){
        if(x > 1) cout << ' ';
        cout << ans[x];
    }
    cout << '\n';
    return 0;
}
```

部分分思路

每个测试点为 4 分。下表列出各做法适用的点组；不同做法有重叠覆盖，分值不能直接相加。

序号	条件与点组	点组分值	做法与复杂度
1	$n \le 5000$；点 $1 \sim 12$	48 分	逐局模拟桌面：用栈保存牌，并记录各点数在栈中的位置。重复点数出现时，弹出它及其右侧的牌。每局每张牌至多入栈、出栈各一次，最后遍历栈累计答案，总时间 $O(n^2)$。
2	性质 A，序列非递减；点 $7 \sim 9, 20$	16 分	相同点数连续出现，打完这一段后不会再收走它。起点在 $last[x]$ 及其左侧的游戏都会保留 x，因此 $ans[x] = last[x]$。时间 $O(n+m)$。
3	$m \le 2$；点 $1 \sim 3, 13, 14$	20 分	设最后一张牌的点数为 y，它在全部 n 局中保留。若还有另一点数 x，则以 x 开始的局最终留下 x, y，以 y 开始的局最终只留下 y，所以 $ans[x]$ 等于 x 在原序列中的出现次数。时间 $O(n+m)$，也适用于 $m = 1$。
4	性质 B，至多 4999 处相邻点数不同；点 $15 \sim 17$	12 分	将连续相同点数压成一段，记段数为 r，则 $r \le 5000$。从同一段内各位置开始的游戏结果相同，故对每个段起点模拟压缩后的后缀，将最终保留点数的答案加上起始段长度。时间 $O(n + r^2)$。
5	$m \le 3000$；点 $18, 19$	8 分	由正文的首牌结论，首牌点数相同的游戏结果相同。先统计每种点数的出现次数，再枚举首牌点数，沿 $last[a_i] + 1$ 分别求出最终保留牌，并按首牌出现次数加权。每条路径至多经过 m 种点数，共 $O(n + m^2)$ 时间。
满分	点 $21 \sim 25$；同时覆盖全部测试点	20 分	将拆分中到达同一起点的游戏局数一起累计，每个起点只处理一次，使用正文的 $O(n+m)$ 做法。

第 4 种做法还自然覆盖点 $1 \sim 12$：段数 $r \le n \le 5000$，合计覆盖 60 分。连续打出同一点数时，第一张之后的牌只会替换桌面最右端的同点数牌，因此压缩等值段不改变结果；加上起始段长度，才计入了这一段内的全部起点。

第 5 种做法覆盖所有 $m \le 3000$ 的点，即点 $1 \sim 19$，共 76 分。若再对非递减序列使用第 2 种做法，还能覆盖点 20，共 80 分。